

QistEngine

One utility bill and one mobile-wallet ledger become a defensible installment offer — for the ~100 million Pakistanis a credit bureau has never heard of.

Repository github.com/sohjjpeg/qistengine

Stack FastAPI · SQLite · Next.js 14 · LightGBM · SHAP

Runs fully offline · two commands

BUILT
BY



Sohaib Amir Bukhari

BS CS · NUST · linkedin.com/in/sohaib-amir-7a89161a6



Asma Imran

BS CS · NUST · linkedin.com/in/asmaimran

■ The problem

Around **100 million adults in Pakistan (~79%)** have no bank account and no credit file. A kiryana shopkeeper with twelve years of trading history was never given a first loan to build a history with — so bureau scoring has nothing to score. The MSME financing gap is roughly **US\$57.8 billion, about 20% of GDP**; fewer than 200,000 of 3.2 million SMEs have access to formal credit.

■ The insight

The excluded are not invisible. They leave a financial trail every month — a **utility bill** (payment punctuality, arrears, reconnections) and an **80M+ mobile-wallet ecosystem** (inflow, volatility, buffer, merchant/QR mix, committee/BC participation). The data to underwrite these borrowers already exists; nobody turns it into a decision an officer can defend. That is the whole product.

■ What it does

From one electricity bill (PDF or scan) and one wallet statement (CSV or PDF export), QistEngine produces:

- a **300–850 score** and risk band;
- a **calibrated probability of default** — a real frequency, not a ranking;
- an **exact reason-code ledger** that reads like a shopkeeper's *khata* and sums, line by line, to the score;
- a **safe monthly "Qist"** installment sized to the applicant's cashflow;
- a **what-if panel** that re-scores live as the officer moves a lever;
- an **adverse-action notice** and a **printable credit memo**.

LOW 720–850

MEDIUM 640–719

HIGH 560–639

VERY HIGH 300–559

■ Who it's for

A microfinance / BNPL loan officer at the branch counter. The interface is a working underwriting tool — an applicant portal, an underwriting queue, an applicant decision page, and a portfolio-analytics view with the model card and a fairness audit — not a demo dashboard.

■ What we built

- **Ingestion.** An OCR cascade for the bill (PDF text layer → Tesseract → clearly-labelled fallback, never fabricated) and a statement parser that normalises JazzCash / EasyPaisa / hand-kept *Digital Khata* exports — tolerant of preamble rows, split debit/credit columns and Rs/comma formatting, with Urdu-and-English transaction classification. It raises a clear error rather than guessing on an unreadable file.
- **26 behavioural features** — cashflow level and volatility, buffer, utility payment discipline, savings behaviour, business tenure, committee participation. Gender, religion, ethnicity, caste, marital status and area proxies are **forbidden as features**; they exist in the synthetic data only so the fairness audit can run.
- **A calibrated LightGBM scorecard** — isotonic calibration; a points-to-double-the-odds scaling where the score is affine in log-odds and SHAP values are additive in log-odds, which is why the ledger foots exactly.
- **A fairness audit on every training pass** — a four-fifths (80%) rule check across groups. Gender and city tier pass; two livelihood groups (daily-wage, ride-hailing) are flagged as genuine cashflow risk, with documented mitigations.
- **A responsible-AI posture throughout** — a "demonstration model / not a regulated credit decision" disclaimer on every decision surface, adverse-action codes for every decline, fully deterministic (seed 42, byte-identical metrics on a re-run), and **no network on the hot path** — no CDN, no LLM call.

ROC-AUC	KS STATISTIC	GINI
0.819	0.488	0.638
BRIER (CALIBRATION)	LOGISTIC BASELINE AUC	BASE DEFAULT RATE
0.099	0.817	14.5%

Calibrated, not just ranked — a predicted 8% PD means 8 in 100, which is what the affordability maths needs. Beats the transparent logistic baseline and separates the tails far better.

■ Honest about the data

This is a prototype. The model is trained on a **synthetic portfolio** generated from a documented data-generating process (archetype cashflow distributions, non-linear repayment dynamics, measurement noise) because no lender hands a student team its repayment book. Everything else — the feature engineering, the scorecard maths, the calibration, the fairness audit, the ingestion pipeline, the officer UI — is production-shaped.

A pilot replaces exactly one thing: the synthetic outcomes, swapped for a partner institution's real repayment history, then recalibrated. The architecture doesn't move.

■ Running it

```
git clone https://github.com/sohjpeg/qistengine.git && cd qistengine && bash run-dev.sh
```

First run bootstraps itself (Python venv, synthetic data, model training, npm install, seed) in ~4 minutes, then starts both servers. App on localhost:3000, API docs on localhost:8000/docs. Six demo profiles are built in — no data to upload. **34 backend tests pass; every dependency is pinned to an exact version.**

DEMONSTRATION MODEL TRAINED ON SYNTHETIC DATA. NOT A REGULATED CREDIT DECISION.